

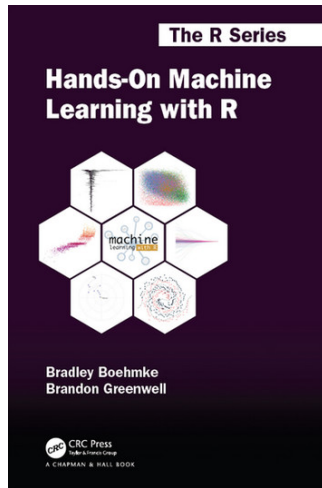
Emergent Digital Technologies

Support Vector Machines (SVM)

Brad Boehmke & Brandon Greenwell

30 de abril de 2026

Hands-On Machine Learning with R



Content has been extracted from *Hands-On Machine Learning with R*, by Boehmke, B. & Greenwell, B.

CRC Press. 2019. Visit <https://bradleyboehmke.github.io/HOML/>.

Introduction to Support Vector Machines

- ▶ SVMs offer a direct approach to binary classification.
- ▶ The goal is to find a **hyperplane** in a feature space that “best” separates two classes.
- ▶ Two main extensions make SVMs powerful in practice:
 1. Loosening the requirement for “perfect” separation.
 2. Using the **kernel trick** to enlarge the feature space for better separation.

R Implementation Prerequisites

- ▶ Key packages for SVM in R:
 - ▶ `kernlab`: The most flexible implementation for fitting SVMs.
 - ▶ `caret`: Used for tuning hyperparameters and pre-processing.
 - ▶ `e1071`: An alternative package for SVM implementation.
- ▶ Interpretability tools:
 - ▶ `vip`: For variable importance plots.
 - ▶ `pdp`: For partial dependence plots.

Optimal Separating Hyperplanes

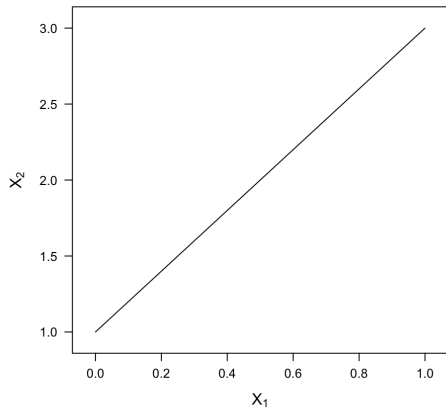
- ▶ A hyperplane in p -dimensional space is defined by:

$$f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p = 0$$

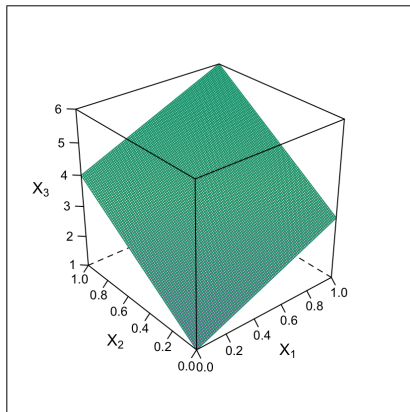
- ▶ For $p = 2$, it's a line; for $p = 3$, it's a plane.
- ▶ The hyperplane acts as a **decision boundary**.
- ▶ Points are classified based on which side of the boundary they fall ($f(X) > 0$ or $f(X) < 0$).

Optimal Separating Hyperplanes (Cont.)

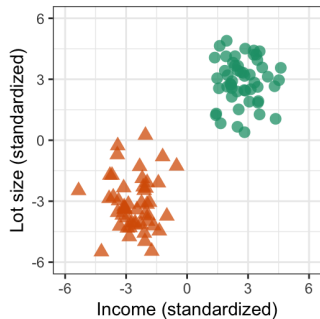
$$f(\mathbf{X}) = 1 + 2X_1 - X_2 = 0$$



$$f(\mathbf{X}) = 1 + 2X_1 + 3X_2 - X_3 = 0$$

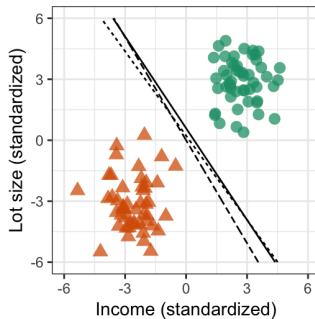


Optimal Separating Hyperplanes (Cont.)



Owns a riding mower?

- Yes
- ▲ No



Owns a riding mower?

- Yes
- ▲ No

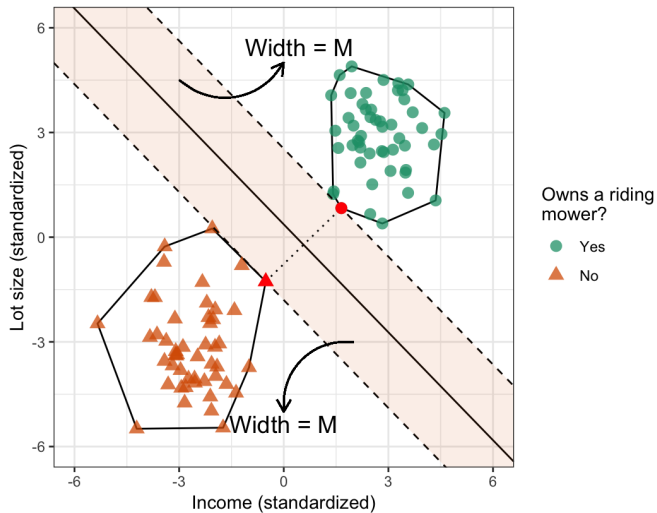
Model

- GLM
- ... HMC
- - - LDA

The Hard Margin Classifier (HMC)

- ▶ If classes are perfectly separable, HMC finds the boundary that provides **maximum separation**.
- ▶ It maximizes the distance to the closest points from either class.
- ▶ This distance is called the **margin (M)**.
- ▶ Geometric construction:
 1. Draw convex hulls around each class.
 2. Find the shortest line segment connecting the hulls.
 3. The perpendicular bisector of this segment is the HMC.

The Hard Margin Classifier (Cont.)



HMC Optimization Problem

The Hard Margin Classifier solves the following quadratic programming problem:

Optimization

Maximize M with respect to $\beta_0, \beta_1, \dots, \beta_p$.

Subject to:

$$\sum_{j=1}^p \beta_j^2 = 1$$

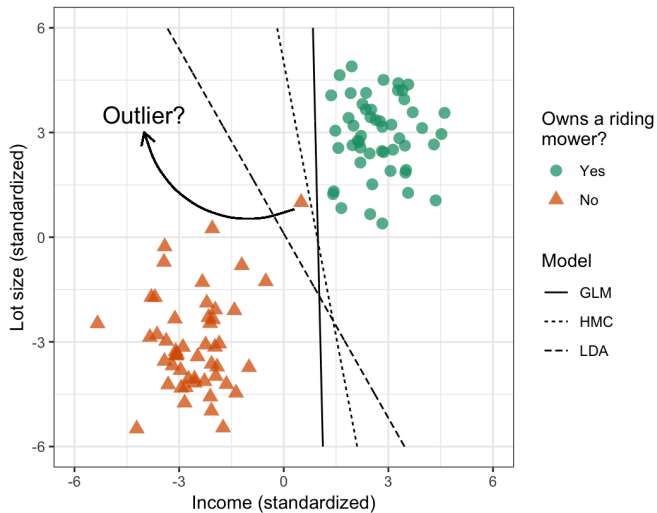
$$y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M, \quad i = 1, \dots, n$$

Note: HMC allows no points on the wrong side of the margin.

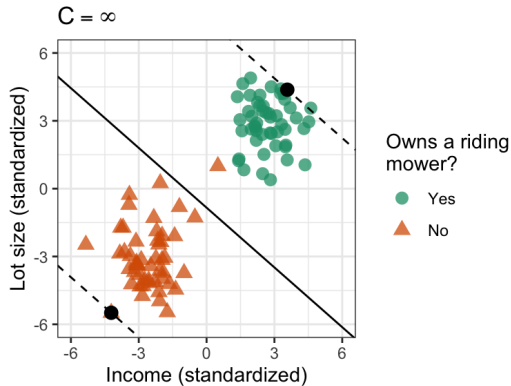
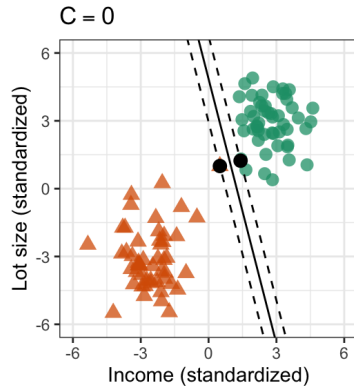
The Soft Margin Classifier (SMC)

- ▶ Hard margins are sensitive to outliers and may not generalize well.
- ▶ SMC allows some points to cross the margin boundaries.
- ▶ Introduces **slack variables** (ξ_i) and a **cost parameter** (C).
- ▶ C represents the allowable “budget” for total overlap.
- ▶ Varying C makes the model robust to outliers.

The Soft Margin Classifier (Cont.)



The Soft Margin Classifier (Cont.)



The Support Vector Machine & Kernel Trick

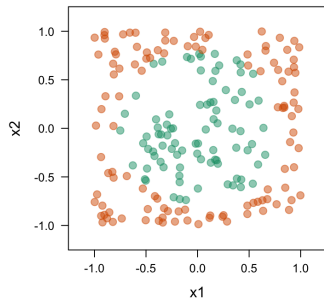
- ▶ Linear boundaries are often too restrictive for complex data.
- ▶ The **kernel trick** enlarges the feature space using basis functions.
- ▶ A hyperplane can separate classes in the *enlarged* space, which becomes a **nonlinear** boundary when projected back to the original space.
- ▶ Example: Adding $X_3 = X_1^2 + X_2^2$ to separate nested circles.

Common Kernel Functions

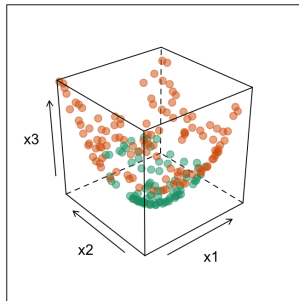
- ▶ **Polynomial:** $K(x, x') = (1 + \langle x, x' \rangle)^d$.
- ▶ **Radial Basis Function (RBF):** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$.
- ▶ **Hyperbolic Tangent:** $K(x, x') = \tanh(k_1 \|x - x'\| + k_2)$.
- ▶ The RBF kernel is extremely flexible and is usually the default starting point in practice.

Common Kernel Functions (Cont.)

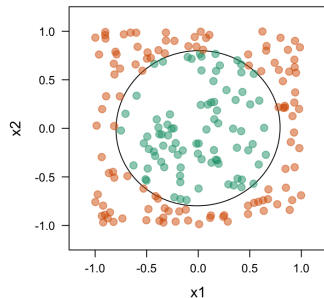
Original feature space



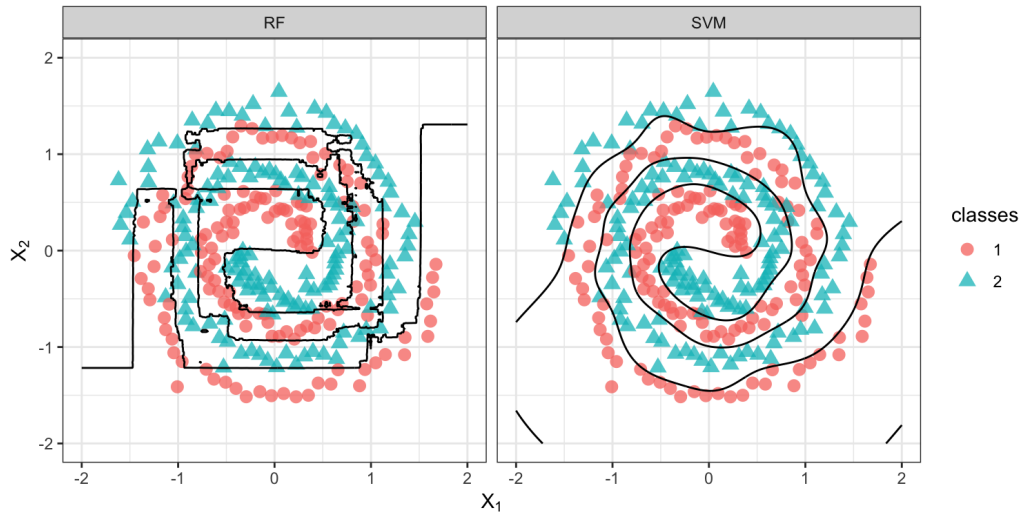
Enlarged feature space



Non-linear decision boundary



Common Kernel Functions (Cont.)



Multinomial Classification

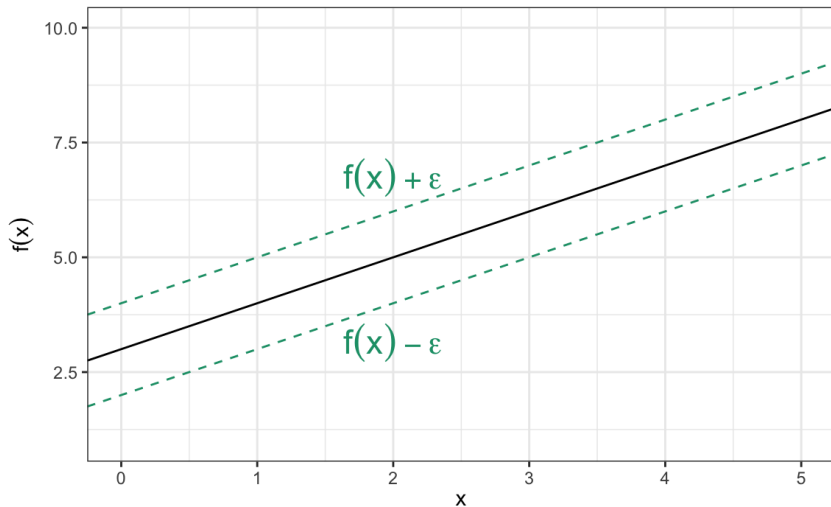
SVM is natively a binary classifier. For > 2 classes, two strategies are used:

- ▶ **One-Versus-All (OVA):** Fit an SVM for each class against the rest. Classify based on the largest margin.
- ▶ **One-Versus-One (OVO):** Fit SVMs for all pairwise combinations. Classify to the class that wins the most competitions.
- ▶ R packages like `kernlab` handle these automatically.

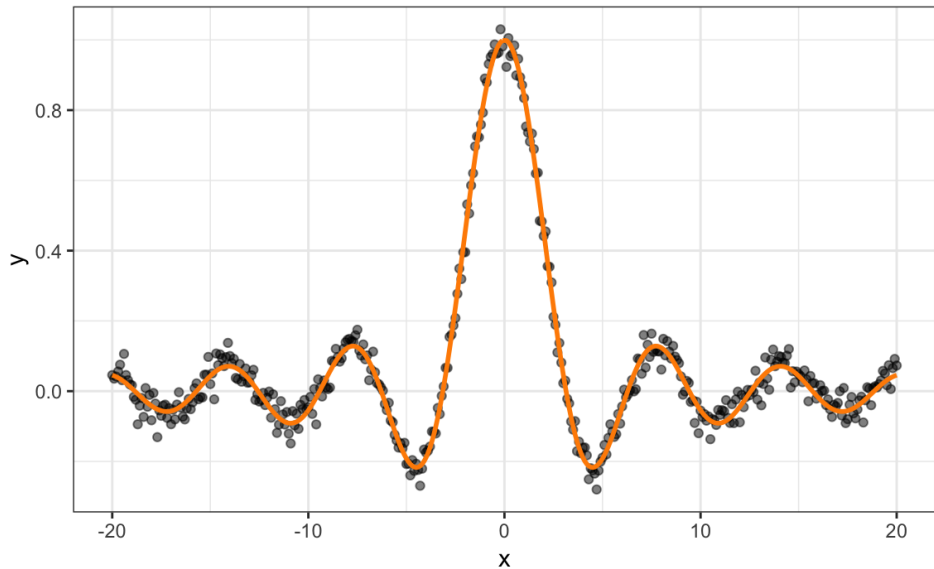
Support Vector Regression (SVR)

- ▶ SVMs extend to continuous outcomes.
- ▶ Uses ε -**insensitive loss**: residuals smaller than ε have zero influence on the fit.
- ▶ It forms a “margin band” around the regression curve.
- ▶ SVR is more robust to outliers than Least Squares because it doesn't square the residuals.

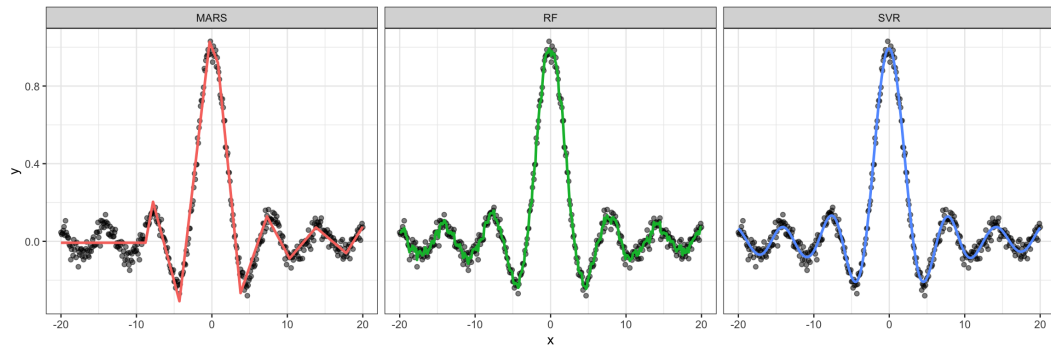
Support Vector Regression (Cont.)



Support Vector Regression (Cont.)



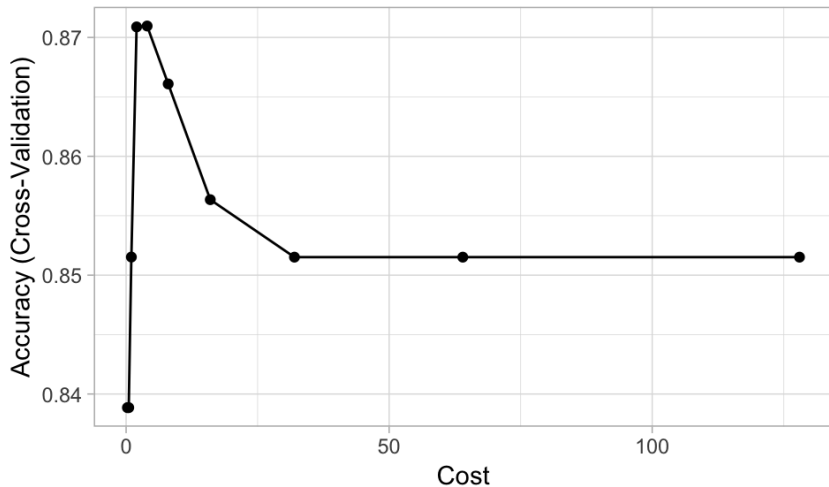
Support Vector Regression (Cont.)



Tuning and Implementation

- ▶ SVMs require tuning hyperparameters like **C** (cost) and σ / γ (kernel parameter).
- ▶ In `caret`, use `train()` with `method = "svmRadial"`.
- ▶ The `sigest()` function in `kernlab` can provide good automated estimates for σ .
- ▶ Cost parameter C is typically searched over an exponentially growing series (e.g., $2^{-2}, \dots, 2^8$).

Tuning and Implementation (Cont.)



Advanced Considerations

- ▶ **Class Imbalance:** Use `class.weights` to assign higher costs to misclassifying the minority class.
- ▶ **Probabilities:** SVMs don't naturally provide probabilities. Set `prob.model = TRUE` or `classProbs = TRUE` to estimate them via Platt scaling.
- ▶ Useful for metrics like AUC (Area Under the Curve).

Feature Interpretation

- ▶ SVMs do not have a built-in feature importance measure.
- ▶ **Permutation Approach:** Use the `vip` package to measure how much a metric (like AUC) drops when a feature is shuffled.
- ▶ **Partial Dependence Plots (PDPs):** Use the `pdp` package to visualize the relationship between the top features and the predicted probability.

The Support Vectors

- ▶ Support vectors are the data points that lie closest to the decision boundary.
- ▶ These specific observations “support” the hyperplane; if they were moved, the boundary would shift.
- ▶ In an RBF or non-linear kernel, these points represent the most difficult cases to classify.
- ▶ Most of the training data (those far from the boundary) have no impact on the final model, making SVMs memory-efficient for prediction.

Slack Variables and the Soft Margin

- ▶ Slack variables ξ_i are introduced to allow for misclassifications in non-separable data.
- ▶ $\xi_i = 0$: The point is on the correct side of the margin.
- ▶ $0 < \xi_i \leq 1$: The point is within the margin but on the correct side of the hyperplane.
- ▶ $\xi_i > 1$: The point is on the wrong side of the hyperplane (misclassified).
- ▶ The optimization goal is to maximize the margin while keeping the sum of ξ_i within the budget C .

The Influence of the Cost Parameter C

- ▶ The parameter C controls the tradeoff between a “clean” boundary and a “wide” margin.
- ▶ **Small C :** Allows more margin violations. This leads to a wider margin and a simpler model (Higher Bias, Lower Variance).
- ▶ **Large C :** Penalizes violations heavily. This leads to a narrow margin that tries to classify every training point correctly (Lower Bias, Higher Variance).
- ▶ Finding the optimal C via Cross-Validation is critical to prevent overfitting.

Tuning the RBF Kernel (γ)

- ▶ The γ (sigma) parameter defines how far the influence of a single training example reaches.
- ▶ **Low** γ : A single point has a “far-reaching” influence. The resulting decision boundary is smooth and less wiggly.
- ▶ **High** γ : Influence is localized. The boundary will stay close to individual points, potentially leading to “islands” around data.
- ▶ High γ values risk overfitting the training data by creating a complex, irregular boundary.

Importance of Data Pre-processing

- ▶ SVMs calculate distances between points to determine the hyperplane and margin.
- ▶ Because of this, SVMs are **not** scale-invariant.
- ▶ Features with larger ranges (e.g., “Income” vs. “Years of Education”) will dominate the distance calculation.
- ▶ **Standardization** (centering to mean 0 and scaling to SD 1) is a mandatory step before training an SVM to ensure all features contribute equally.

Final Thoughts: Advantages and Disadvantages

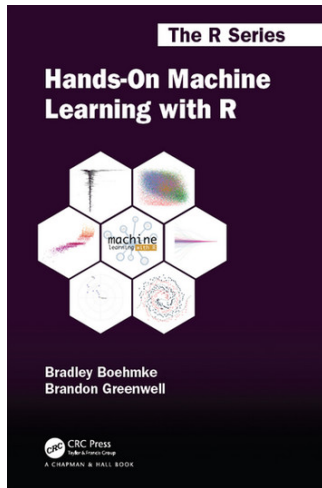
Advantages

- ▶ Global optimum guaranteed (convex optimization).
- ▶ Robust to outliers (via C).
- ▶ Extremely flexible for nonlinear data.

Disadvantages

- ▶ Slow to train on “tall” data ($n \gg p$).
- ▶ High memory/parameter usage.
- ▶ Probability estimates require extra steps.

Hands-On Machine Learning with R



Content has been extracted from *Hands-On Machine Learning with R*, by Boehmke, B. & Greenwell, B.

CRC Press. 2019. Visit <https://bradleyboehmke.github.io/HOML/>.